Poster: SurakshaNet: Privacy-First, Edge-Deployed Multilingual Abusive Language Detection

Aamod Jain*
Indian Institute of Technology Ropar
Punjab, India
2023csb1092@iitrpr.ac.in

Pratham Garg*
Indian Institute of Technology Ropar
Punjab, India
2023csb1147@iitrpr.ac.in

Geeta Yadav
Indian Institute of Technology Ropar
Punjab, India
geeta@iitrpr.ac.in

ABSTRACT

Traditional abusive language moderation often relies on cloud-based APIs, raising privacy concerns and introducing network latency. We present *SurakshaNet*, an edge-deployed system for real-time multilingual toxicity detection on web messaging platforms, with a prototype implemented for WhatsApp Web. By combining INT8 static quantization with ONNX Runtime Web and WebAssembly (WASM), we reduce a 471 MB transformer model to 113 MB (about 76%) while incurring less than 0.11% accuracy loss. Implemented natively as a Chrome Extension using a Manifest V3 Service Worker, *SurakshaNet* achieves a median inference latency of 28 ms and ensures that raw user data never leaves the device. We further extend the system with Hinglish training data, a gated conversational-context window, and child-safety features (parental control, harmful-message hiding, and screenshot evidence collection), and show that the local model matches or outperforms evaluated cloud LLMs on several conversational slices. Finally, we add a 105,000-thread, five-class benchmark in English, Hindi and Hinglish, on which thread-aware models reach 97.9% test accuracy with a 118 MB INT8 model.

CCS CONCEPTS

• **Security and privacy → Software and application security**.

KEYWORDS

Abusive language detection, Privacy-preserving AI, Edge computing

ACM Reference Format:
Aamod Jain*, Pratham Garg*, and Geeta Yadav. 2026. Poster: SurakshaNet: Privacy-First, Edge-Deployed Multilingual Abusive Language Detection. In *The 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys Companion '26), June 21–25, 2026, Cambridge, United Kingdom.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3812835.3814864

*Equal contribution.

1 INTRODUCTION

Malicious actors increasingly exploit social media platforms to spread harmful content [7], creating toxic online spaces where women, children, and other marginalized groups are frequently targeted. As gender-based harassment grows in code-mixed Indian contexts (Hinglish/Hindi) [4], accurate and timely abuse detection becomes essential for safe user interaction. There is an urgent need for proactive, privacy-preserving moderation tools. To address this, we develop *SurakshaNet*, a browser-based extension for real-time detection of harmful and abusive conversations.

Our contributions are:

- A lightweight web extension for in-browser abuse detection on web apps, with parental controls, harmful-message hiding, and automatic screenshot evidence collection.
- A multilingual abuse detection pipeline with a size-optimized, edge-deployable model and forensic evidence support.
- A Hinglish dataset derived from MACD and a conversation-aware (gated window) inference strategy, benchmarked against cloud-based LLMs.
- A five-class, context-thread dataset (benign, friendly sarcasm, hostile abuse, sexual harassment, predatory grooming) and thread-aware models trained on it.

2 SYSTEM IMPLEMENTATION

2.1 Detection Pipeline

The core intelligence follows a 5-step pipeline, as shown in Figure 1, beginning with multilingual abusive-language datasets and proceeding through model training. We trained our model on a combined dataset of 42,487 records, consisting of the MACD Hindi subset [4] and the Davidson English (EN) dataset (90% training split)[1]. The MACD dataset is a large-scale, human-annotated multilingual corpus from ShareChat, balanced with 49% abusive content. This combined training ensures the model is robust to both Hindi and English offensive language. A fine-tuned `MiniLM-L12-v2` is aggressively compressed using ONNX quantization [5], shrinking its RAM footprint to approximately 89 MB. This substantial reduction enables efficient browser-native execution through `onnxruntime-web`, making real-time edge inference practical.

The model runs in a browser Service Worker, allowing for zero-network-dependency classification. A `MutationObserver` script monitors platforms like WhatsApp Web by injecting warning banners upon detecting toxic nodes. The optimized model is deployed as a Chrome extension, where ONNX-based in-browser inference

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MobiSys Companion '26, June 21–25, 2026, Cambridge, United Kingdom
© 2026 Association for Computing Machinery.
ACM ISBN 979-8-4007-2711-5/2026/08…$15.00
https://doi.org/10.1145/3812835.3814864

[1] https://github.com/t-davidson/hate-speech-and-offensive-language

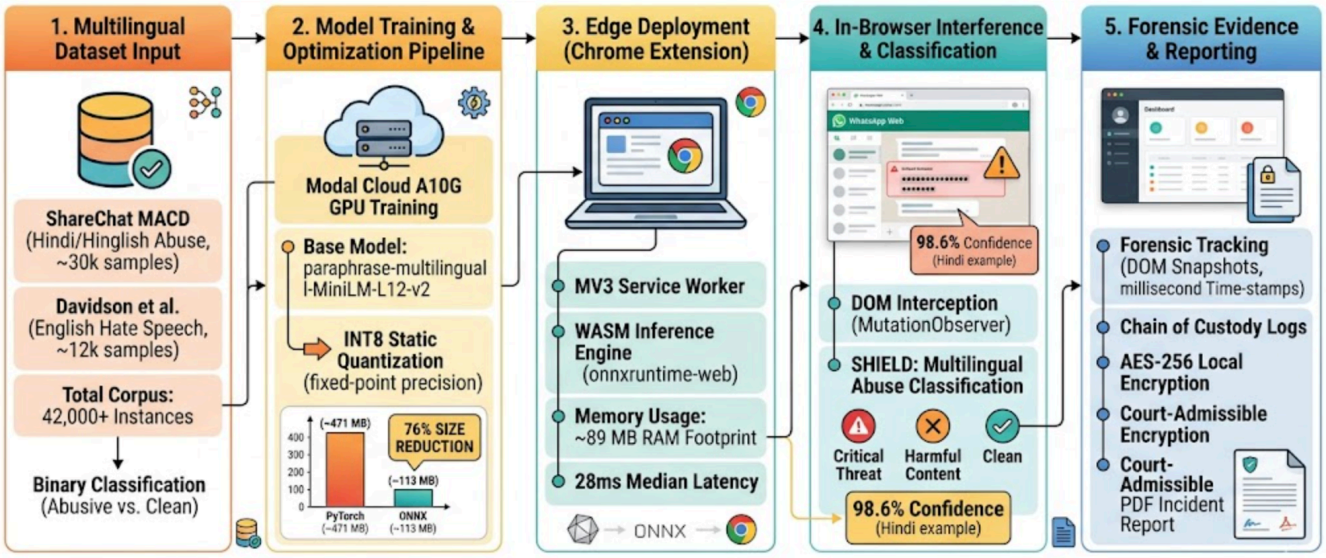


Figure 1: End-to-end architecture of *SurakshaNet*, including multilingual dataset preparation, transformer model training, INT8 quantization, Chrome-based edge deployment via ONNX Runtime Web and WASM, real-time abuse detection through DOM interception, and forensic evidence logging.

supports fast, lightweight, and privacy-preserving detection of harmful content. This design makes the framework suitable for edge environments by avoiding cloud inference, reducing latency, and protecting sensitive user communication. Finally, the framework supports forensic evidence collection through DOM snapshots, chain-of-custody logs, AES-256 local encryption, and court-admissible incident report generation.

As shown in Figure 2, the system identifies both "Harmful" and "Critical" threats in multilingual contexts. It correctly classifies Hinglish/H slang (e.g., catching abuse with 98.6% confidence) while maintaining a low false-positive rate for neutral conversational Hindi. The source code is available at the Project Repository[2].

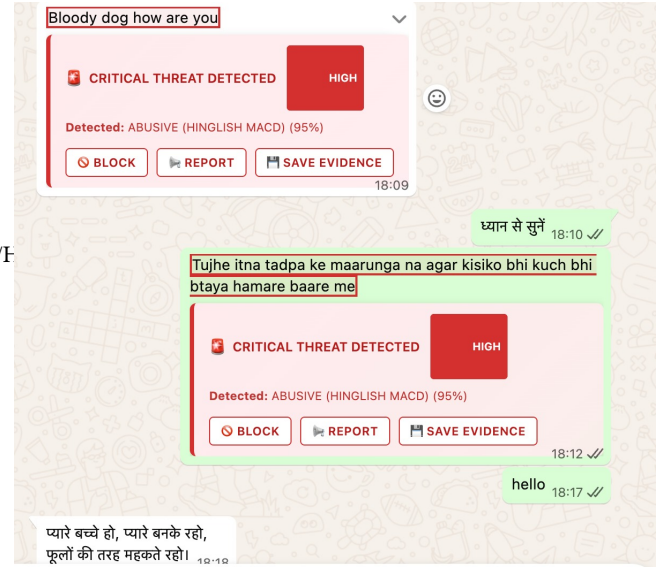


Figure 2: Examples of *SurakshaNet* detecting abusive messages in real time on WhatsApp Web, showing severity-aware alerts.

2.2 Child-Safety Enhancements

To move *SurakshaNet* from a basic abusive-language detector toward a more complete child-safety system, we added the following features.

Dual login architecture. The extension supports two authentication modes: a *parent login*, through which parents set up an account for their child's safety, and a *normal user login*, through which general users set up accounts for themselves.

Child Protection Mode. This mode prevents the child from disabling or circumventing the extension. While it is enabled, security-sensitive operations such as disabling the extension or logging out require the **master password** configured by the parent. This matters because the protection mechanism should not be easily removable by the user it is intended to protect.

Automatic screenshot evidence. When an abusive-language incident is detected, the extension automatically captures a screenshot of the relevant WhatsApp chat window. The screenshot is stored with the incident record and shown on the parent dashboard, giving context beyond the flagged text. An incident record therefore contains the flagged message text, the chat-window screenshot, sender information, and a timestamp.

[2]https://github.com/PrathamGarg1/surkashanet

Harmful message visibility toggle. Parents can choose to visually conceal messages classified as harmful from the child instead of merely flagging them, adding a preventive layer on top of detection.

Overall workflow. The parent authenticates, configures the child-safety settings, and enables Child Protection Mode. The child then uses the browser through the normal user session while messages are analysed locally by the detection model. Harmful messages are flagged or hidden according to the configured policy, and the relevant chat window is captured as evidence. The parent later reviews incidents and evidence through the parental interface, and security-sensitive operations remain protected by the master password.

3 EMPIRICAL EVALUATION

3.1 Quantization and Hindi/English Accuracy

As shown in Table 1, the quantized model achieves a median latency of 28 ms and a compact memory footprint of approximately 89 MB, making it suitable for resource-constrained real-time inference. Our model achieves an accuracy of 84.66% on Hindi-only content (Test A) and 88.01% on Hindi+English content (Test B). The MACD paper reports 86.34% accuracy using a fine-tuned XLM-R Base model on Hindi-only data. While our edge model shows a 1.68% performance trade-off on Hindi-only tasks relative to this baseline, XLM-R Base (with approximately 270–278 million parameters) is infeasible for in-browser deployment due to its prohibitive memory footprint, far exceeding the RAM constraints of a browser runtime. Suitability for browser-native, privacy-first edge inference was the primary design constraint governing model selection. Crucially, for the multilingual Hindi+English setting, our model achieves 88.01%—surpassing the MACD baseline (which reports no English-inclusive accuracy)—demonstrating stronger generalization across languages. The observed accuracy degradation relative to the non-quantized version of our own model is negligible, remaining below 0.11%. In addition, the model size is reduced from 471 MB to 113 MB, a 76% reduction, substantially improving storage efficiency and portability. All incidents are timestamped to millisecond precision and hashed (SHA-256) to ensure tamper-proofness. In the future, we will explore blockchain-based mechanisms to securely store forensic evidence in the cloud.

Table 1: Post-Quantization Performance Summary

Metric	Value
Inference Latency	28 ms
RAM Footprint	~89 MB
Accuracy – Original Paper (XLM-R, Hindi)	86.34% (not browser-deployable)
Accuracy (Test A: SurakshaNet, Hindi-only)	84.66%
Accuracy (Test B: SurakshaNet, Hindi+Eng)	88.01% (no baseline from paper)
Performance Delta (Hindi-only vs. [4])	−1.68%
Quantization Loss	<0.11% (vs. non-quantized model)
Size Reduction	76% (471 MB → 113 MB)

4 EXTENSION: HINGLISH DATA AND CONVERSATIONAL CONTEXT

The sections above describe *SurakshaNet* as first presented. The rest of this report extends it in three steps: a Hinglish-aware encoder with a conversational-context window (this section), a five-class thread benchmark (Section 5), and thread-aware models trained on it (Section 6). Numbers in this and the following sections refer to the retrained extension model, not to the first model of Table 1.

4.1 Hinglish Dataset and Retrained Encoder

A major limitation of standard Hindi/English abusive-language datasets is their mismatch with how many people in India actually write online: Hindi and English are frequently mixed within a sentence, often in Romanized Hindi (Hinglish). To address this, the MACD dataset was converted into a Hinglish dataset using a Qwen-based LLM pipeline. For comparison, we also implemented and evaluated the data-generation strategies described in CodeMixBench [13], a benchmark for evaluating language models on code-mixed text, which also proposes generating synthetic code-mixed text by combining word substitution with LLM-based prompting. The extension encoder was retrained on 58,929 messages (MACD Hindi 20,183, Davidson English 19,826, Hinglish 18,920) and tested on 15,575 held-out messages. Dynamic INT8 quantization shrinks it from 470.6 MB to 118.4 MB (75%), and accuracy is unchanged within 0.15 points on every test set: 85.1% on Hindi, 84.4% on Hinglish, 96.7% on English and 86.5% on the mixed set (Figure 3).

4.2 Evaluation Protocols

The complete evaluation test set was frozen before benchmarking so that results are directly comparable across all methods. The final benchmark uses 6,368 Hinglish messages, a classification threshold of 0.5, and the INT8-quantized MiniLM as the primary model for the task of abusive vs. non-abusive detection. The test data is divided into conversation slices: single messages (6,368), split narratives (1,500), benign conversations (800), and abusive messages appearing after harmless small talk (800). In the split-narrative and small-talk slices all positive examples are abusive, so accuracy on these slices equals recall. We compare several ways of handling conversational context (Table 2). The LLM experiments were used for benchmarking only: the *SurakshaNet* extension makes no Amazon Bedrock API calls and the shipped system uses the locally deployed INT8 model.

4.3 Conversational Results

Abuse is often spread over several messages: the abusive slur occurs outside the final turn in 1,468 of the 1,500 split-narrative cases. Scoring only the current message detects 28.9% of them; scoring the whole conversation raises this to 79.3% (Figure 3, right). Whole-chat scoring, however, lowers accuracy on abuse after small talk to 62.3%. The gated approach, which is the configuration selected for deployment, detects 76.1% of split narratives while keeping 82.9% accuracy when abuse follows small talk and 83.0% on benign conversations. For single messages the abusive-class F1 score is 83.6%.

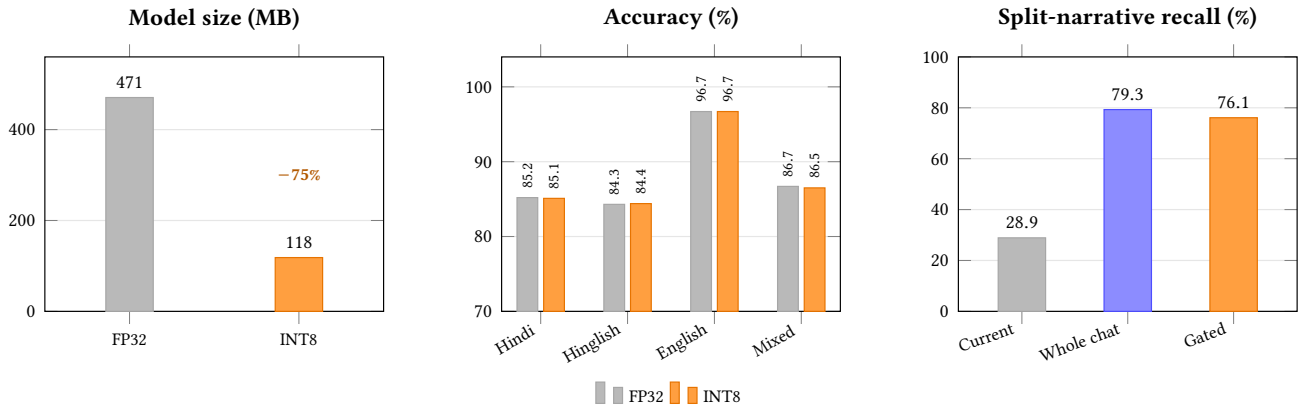


Figure 3: Extension model. Left: INT8 quantization cuts the model size by 75%. Centre: test accuracy before and after quantization. Right: share of split-narrative abuse detected with the current message only, the whole chat, and the gated window shipped in the extension (frozen Hinglish test set).

Table 2: Evaluation protocols used for conversational abusive-language detection.

Setup	Description
Paper protocol	Only the current message is classified. This follows the one-instance setup used for the CodeMixBench-based evaluation.
LLM window	The full conversation history is concatenated into one string and classified as a single instance.
Live LLM	One-shot evaluation with Amazon Bedrock models at temperature zero; the model returns only `abusive` or `non-abusive`.
Shipped gate	The conversation window is evaluated only when the latest turn has fewer than eight tokens and history exists. The final score is the maximum of the current-message and conversation-window scores.

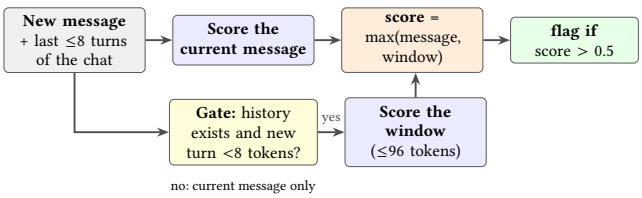


Figure 4: The gated conversational window used by the extension. Short replies, whose meaning depends on the chat, are scored together with the preceding turns.

4.4 Comparison with Cloud-Based LLMs

The same conversational examples were scored with two Amazon Bedrock models, **Nova Micro** and **Qwen3 32B**, using a one-shot prompt at temperature zero (Figure 5). With the full conversation window the local INT8 model detects 77.5% of split narratives, against 65.0% for Qwen3 32B and 40.0% for Nova Micro, and it is ahead of Nova Micro on abuse after small talk and on benign chats. On isolated insults, local recall (78.0%) is on par with Qwen3 32B (76.0%) and well above Nova Micro (40.8%). The subsets are small (20 to 80 examples per slice, Qwen3 32B scored only on the split and insult slices, where the local model scores 80.0% and 78.0% on the same examples), so we read these results as "competitive without any cloud dependency" rather than as a ranking.

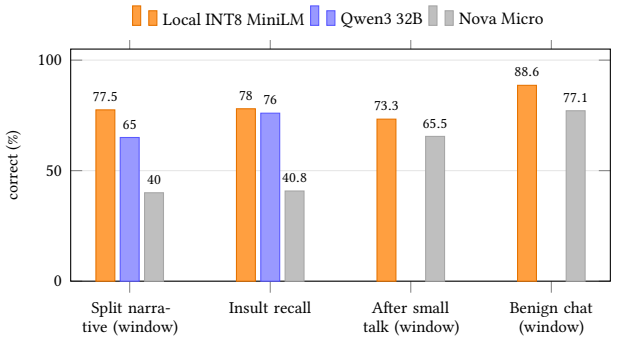


Figure 5: Local INT8 MiniLM versus cloud LLMs on conversational slices (percent correct, higher is better; subsets of 20–80 examples). Qwen3 32B was scored on the first two slices only.

5 EXTENSION: FIVE-CLASS THREAD DATASET

5.1 Why a new dataset

Binary datasets classify one message at a time, yet the same line ("try using your head") can be friendly teasing or a real attack depending on what came before (Figure 6), and grooming or harassment is defined by who is talking to whom. We therefore built a dataset in which the *label belongs to the last message given the earlier ones.* It has five classes (Table 3) chosen from prior taxonomies [1, 3, 6, 10, 12] and aimed at women and children.

Table 3: The five classes. The last-message rule is what the generation prompt asks for.

Id and class	Audience	Last-message rule in the prompt	Basis
0 benign	peers, adults, minors	Normal message. Includes hard negatives: parent–child, teacher–student, harmless surprise secrets, consensual flirting.	HASOC non-offensive [6]
1 friendly_sarcasm	peers	Playful, clearly affectionate teasing; the friends are laughing.	Van Hee et al. [10]
2 hostile_abuse	peers, adults	Hostile personal insult meant to hurt. No slurs about caste, religion, gender, ethnicity or disability; no sexual content.	Waseem et al. [12], MACD
3 sexual_harassment	adult woman (75%) or man (25%)	Unwanted suggestive remark or pressure; not graphic; the target deflects earlier.	Van Hee et al., Guest et al. [3]
4 predatory_grooming	minor (age 10–15)	Secrecy from parents, moving to a private app, meeting alone, age probing. No sexual content.	Banko et al. [1]

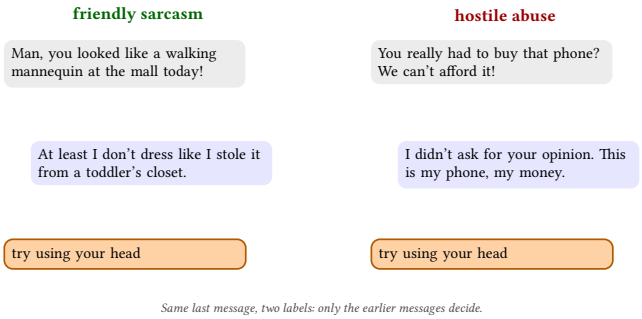


Figure 6: Two generated English threads with the same last message. The earlier messages decide whether it is friendly sarcasm or hostile abuse.

5.2 Generation pipeline

Figure 7 gives the full flow. A grid of 3 scripts (English, Hindi in Devanagari, Hinglish in Roman letters), 4 lengths (3, 5, 7, 10 messages) and 5 classes gives 60 cells of 1,750 threads each, 105,000 in total. For every thread a seeded sampler picks a class-specific subtype (31 in total), a topic (40), two names from a pool of 30, the audience and, where relevant, the target's gender and the child's age. A prompt built from this scenario asks Qwen3-32B [8] on Amazon Bedrock to return exactly k alternating messages as JSON, where the last message follows the class rule. Rule-based validators then check the turn count, refusals, turn length, duplicates, script, and profanity or explicit words outside the hostile class, and sexualised words in grooming threads. In the last generation round, 98.6% of attempts passed. Accepted threads are de-duplicated and split 80/10/10 (83,672 train, 10,765 validation, 10,563 test).

Same-last-line pairs. About half of the friendly_sarcasm and hostile_abuse threads end in an identical line from a bank of 36 lines per script, so only the earlier messages carry the label (Figure 6). Pair lines are split by line, so no pair line appears in two splits; the test set holds 2,337 such threads.

Record format. Each record has 17 columns (Table 4). The dataset is balanced: 21,000 threads per class, 35,000 per script and 26,250 per length.

Table 4: The 17 columns of a record (`jsonl`).

Column	Meaning
`thread_id`, `split`	unique id; train / val / test
`length`, `n_context`	messages in the thread (3/5/7/10); earlier messages
`script`	english / hindi_devanagari / hinglish
`label`, `label_id`	one of the five classes; 0–4
`subtype`, `topic`	scenario (31 subtypes, 40 topics)
`audience`	peers / adult / minor, with gender where relevant
`target_gender`, `child_age`	woman/man or girl/boy; age 10–15 (grooming)
`pair_id`	set for same-last-line threads
`speakers`	the two speaker names, alternating
`turns`, `last_turn`	the messages in order; the final message
`source`	generator tag (Qwen3-32B via Bedrock, synthetic)

6 EXTENSION: THREAD-AWARE MODELS

Figure 8 summarises the experiments. All models use the multilingual MiniLM-L12 encoder [9, 11] with a 5-way output.

Thread model (A). The input is the last message followed by the earlier messages, most recent first, separated by `[TURN]` (256 tokens maximum). It is fine-tuned for 2 epochs (learning rate 4×10^{-5}, batch 32, fp16) with the word-embedding table frozen, on one NVIDIA T4 GPU (AWS g4dn.xlarge, 19.8 minutes, $0.18), and then exported to INT8. **Last-message-only model (B)** uses the same recipe but sees only the final message. **Frozen encoder + GRU (C)** encodes each line once (prefix `in:` or `out:`, 384 numbers), runs a one-layer GRU (hidden size 128) over the vectors, and has only 198,021 trainable weights.

Metrics. *Accuracy* is the share of correctly classified test threads; *macro-F1* is the mean of the five per-class F1 scores. All numbers below are on the 10,563 held-out synthetic test threads.

Table 5: Five-class results on the synthetic test set.

Model	Size	Acc. (%)	Macro-F1
A. Thread model (FP32)	470.8 MB	97.96	0.980
A. Thread model (INT8)	**118.2 MB**	**97.88**	**0.979**
C. Frozen encoder + GRU (INT8)	118 MB + 0.8 MB	94.91	0.950
B. Last message only	470.8 MB	83.18	0.828

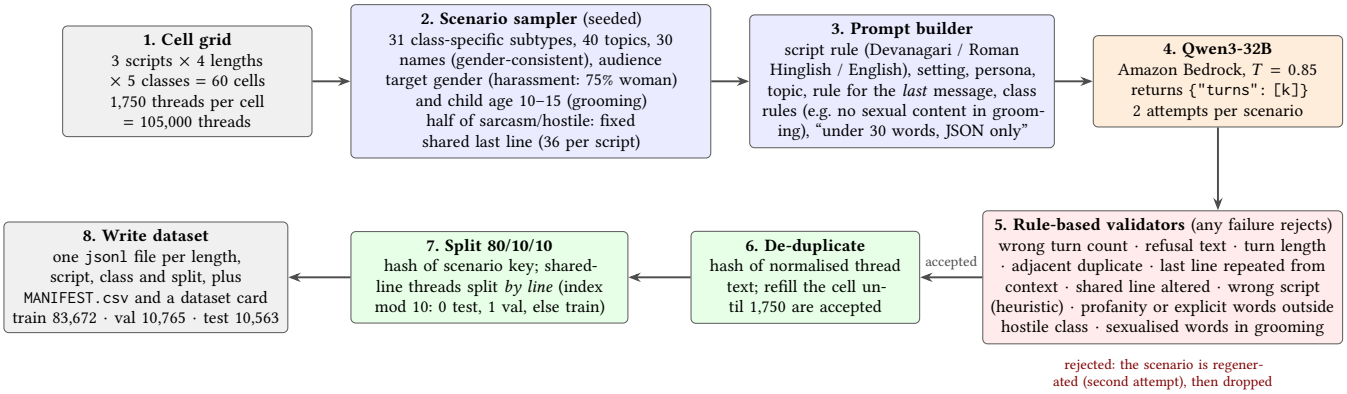


Figure 7: How the dataset was generated. Steps 1–4 create a scenario and a thread; step 5 rejects threads that break a rule; steps 6–8 de-duplicate, split and write the dataset.

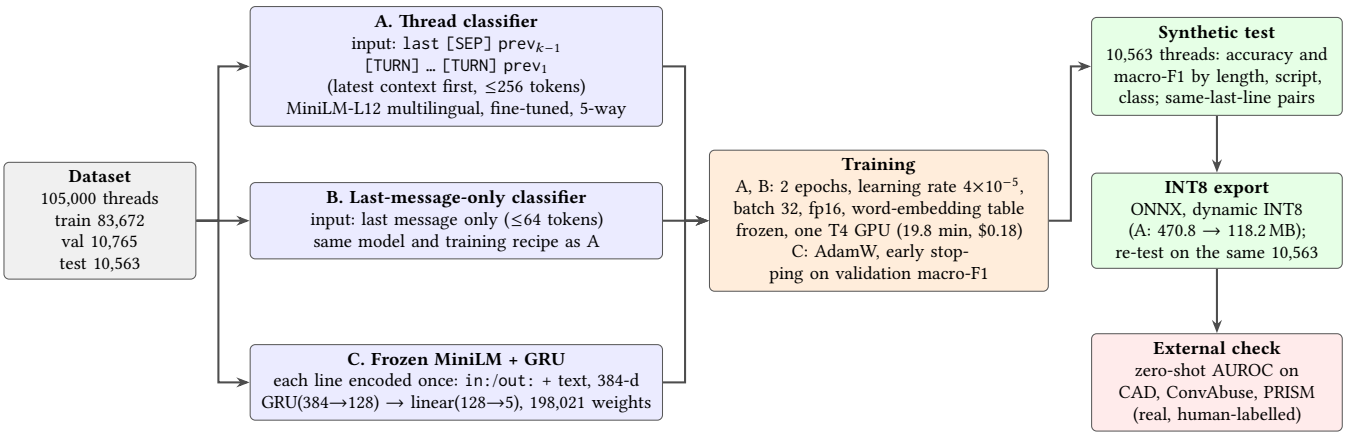


Figure 8: Models, training and evaluation. A and B are fine-tuned end to end; C keeps the encoder frozen and trains only a GRU [2] over per-line vectors.

Thread awareness. Seeing the earlier messages lifts accuracy from 83.2% to 98.0% (Table 5), and the thread model stays above 96% at every length (Figure 9, left), while the last-message-only model gets worse as threads grow. On the same-last-line threads, which a last-message-only model cannot separate by construction, the INT8 thread model is correct 97.5% of the time.

Compact model. INT8 quantization shrinks the thread model by 75% for a loss of 0.08 points. The frozen-encoder variant needs only 198,021 trained weights and reaches 94.9%, which makes it attractive when training budget is small. Per script the INT8 thread model scores 99.3% on English, 97.5% on Hindi and 96.9% on Hinglish, and per-class recall lies between 96.3% and 99.8% (Figure 9, right). With a native single-thread run, a 5-message thread takes a median of 28.3 ms.

7 FINAL DEPLOYED CONFIGURATION

Based on the complete evaluation, the current *SurakshaNet* deployment uses the **INT8 MiniLM encoder with the gated conversational window**, which achieves:

- **84.2%** accuracy on single-message classification,
- **76.1%** recall on split-narrative conversations,
- **82.9%** accuracy when abusive content appears after small talk, and
- **83.0%** accuracy on benign conversations.

The deployed extension does not send user conversations to Amazon Bedrock or any other cloud-based LLM; the Bedrock experiments were conducted solely for benchmarking, and the production inference path remains local, preserving the privacy-first design. The five-class thread models are research artifacts; the extension itself still uses the binary model.

Limitations and next steps

The five-class data are generated by an LLM and checked by automatic rules; no human annotators reviewed them, so the scores in Section 6 describe the synthetic distribution. A first zero-shot check of the synthetic-trained models on real conversation data (CAD, ConvAbuse, PRISM) gave AUROC between 0.47 and 0.74, so we see the dataset today as a controlled benchmark and as training

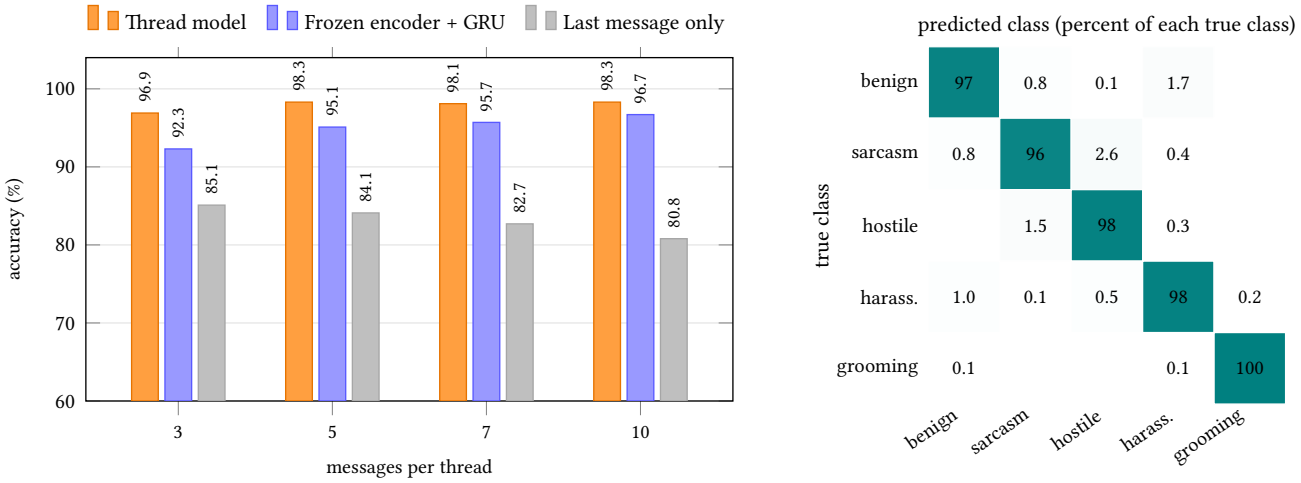


Figure 9: Five-class results on the synthetic test set. Left: accuracy by thread length. Right: confusion matrix of the INT8 thread model (row-normalised, percent).

material to be mixed with real data. Our Hinglish text is machine-converted and less code-mixed than human Hinglish, the cloud-LLM comparison uses small subsets, and latency of the five-class models was measured natively rather than in the browser. The child-safety features of Section 2.2 are described but not evaluated here. Next steps are adding real threaded data with held-out evaluation, obtaining human Hinglish threads, and measuring the INT8 thread models in the browser. The dataset contains synthetic abusive, harassing and grooming text and no real people; grooming threads use only non-sexual cues, and it should be used only to build protective classifiers.

8 CONCLUSION

SurakshaNet demonstrates that proactive, multilingual moderation is technically viable on the edge. By combining low-latency local inference, conversational context handling, parent-controlled security settings, harmful-message hiding, and automatic forensic evidence collection, the system moves beyond a standalone text classifier toward a practical, privacy-first child-safety tool in the browser. The five-class thread benchmark shows that models which use conversation context, including a 118 MB INT8 model, reach 97.9% accuracy on held-out synthetic threads, which gives a clear route to context-aware detection on the device.

REFERENCES

[1] Michele Banko, Brendon MacKeen, and Laurie Ray. 2020. A Unified Taxonomy of Harmful Content. In *Proceedings of the Fourth Workshop on Online Abuse and Harms (WOAH)*.

[2] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[3] Ella Guest, Bertie Vidgen, Alexandros Mittos, Nishanth Sastry, Gareth Tyson, and Helen Margetts. 2021. An Expert Annotated Dataset for the Detection of Online Misogyny. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

[4] Vikram Gupta, Sumegh Roychowdhury, Mithun Das, Somnath Banerjee, Punyajoy Saha, Binny Mathew, hastagiri prakash vanchinathan, and Animesh Mukherjee. 2022. Multilingual Abusive Comment Detection at Scale for Indic Languages. In *Advances in Neural Information Processing Systems*, Vol. 35. 26176–26191.

[5] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.

[6] Thomas Mandl, Sandip Modha, Prasenjit Majumder, Daksh Patel, Mohana Dave, Chintak Mandlia, and Aditya Patel. 2019. Overview of the HASOC track at FIRE 2019: Hate Speech and Offensive Content Identification in Indo-European Languages. In *Proceedings of the 11th Forum for Information Retrieval Evaluation (FIRE)*.

[7] Binny Mathew, Punyajoy Saha, Seid Muhie Yimam, Chris Biemann, Pawan Goyal, and Animesh Mukherjee. 2021. HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 17 (May 2021), 14867–14875. https://doi.org/10.1609/aaai.v35i17.17745

[8] Qwen Team. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[9] Nils Reimers and Iryna Gurevych. 2020. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[10] Cynthia Van Hee, Gilles Jacobs, Chris Emmery, Bart Desmet, Els Lefever, Ben Verhoeven, Guy De Pauw, Walter Daelemans, and Véronique Hoste. 2018. Automatic detection of cyberbullying in social media text. *PLOS ONE* 13, 10 (2018).

[11] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33.

[12] Zeerak Waseem, Thomas Davidson, Dana Warmsley, and Ingmar Weber. 2017. Understanding Abuse: A Typology of Abusive Language Detection Subtasks. In *Proceedings of the First Workshop on Abusive Language Online*.

[13] Yilun Yang and Yekun Chai. 2025. CodeMixBench: Evaluating Code-Mixing Capabilities of LLMs Across 18 Languages. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 2139–2169. https://doi.org/10.18653/v1/2025.emnlp-main.109